Facto - Documentación técnica

PySquad - Equipo 02 - Reto AI Invoice Processor - NEPXA AI Ops Challenge 2026

1. Resumen

Facto es una aplicación web que recibe facturas en PDF o imagen, extrae sus datos con código (pypdf y OCR Tesseract), las valida con un cruce de tres vías contra su orden de compra y su contrato (con sus tarifas pactadas), asigna un estado con reglas de negocio deterministas, registra una causación por cada hallazgo y actualiza el estado de la orden de compra. La IA redacta la explicación de cada resultado y responde las preguntas del contador. La IA no lee las facturas ni decide estados.

2. Problema

El área financiera revisa factura por factura contra contratos y órdenes de compra, a mano y bajo la presión del cierre de mes. Un solo número mal cruzado cuesta dinero real: facturas duplicadas, IVA mal liquidado, precios por encima de lo pactado o facturación que supera el tope del contrato.

3. Arquitectura

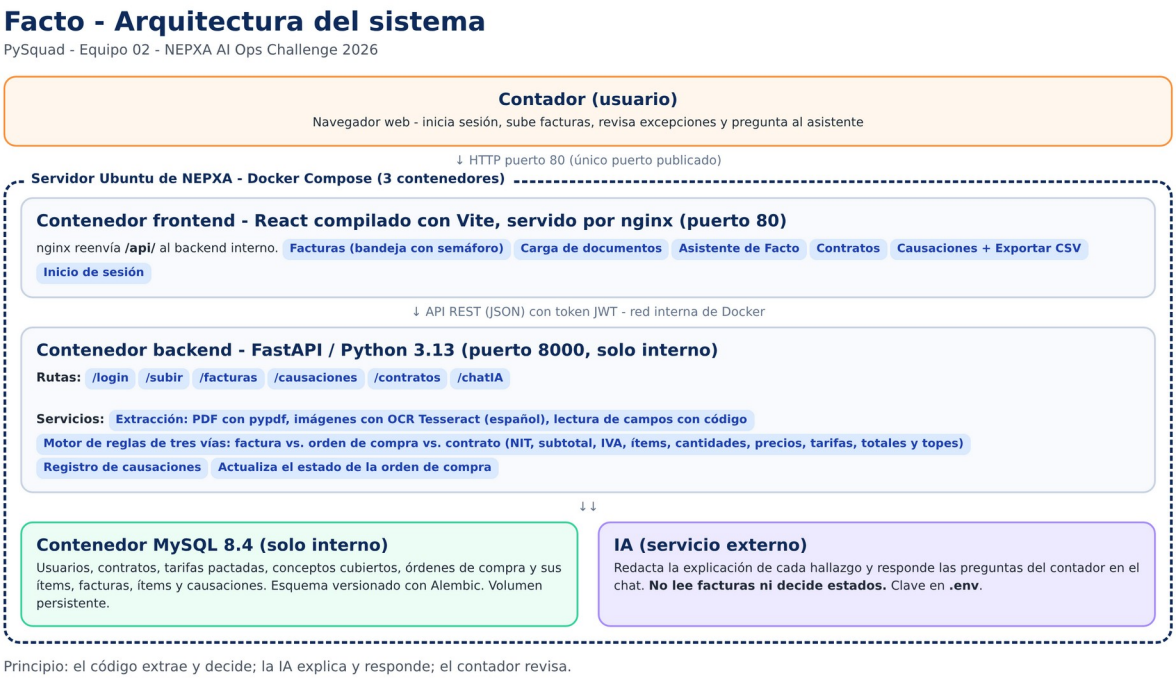


Figura 1. Arquitectura de Facto en el servidor de NEPXA

Componente	Tecnología	Responsabilidad
Frontend	React + Vite, servido por nginx	Inicio de sesión, facturas con semáforo, detalle con causaciones, contratos, carga de documentos, causaciones con exportación CSV y asistente
Backend	FastAPI (Python 3.13), SQLAlchemy, Alembic	API REST con JWT, extracción, validación de tres vías, registro de causaciones, estado de la orden de compra, llamadas a la IA
Extracción	pypdf y Tesseract OCR (español)	Texto de PDF e imágenes; los campos se leen con expresiones regulares
Base de datos	MySQL 8.4	Usuarios, contratos, tarifas, conceptos, órdenes de

Componente	Tecnología	Responsabilidad
		compra y sus ítems, facturas, ítems y causaciones
IA	Servicio de IA externo	Explicación de hallazgos y chat de consultas
Despliegue	Docker Compose	Tres contenedores: frontend, backend y db

4. Flujo de funcionamiento



Figura 2. De la factura a la causación

4.1 Secuencia técnica

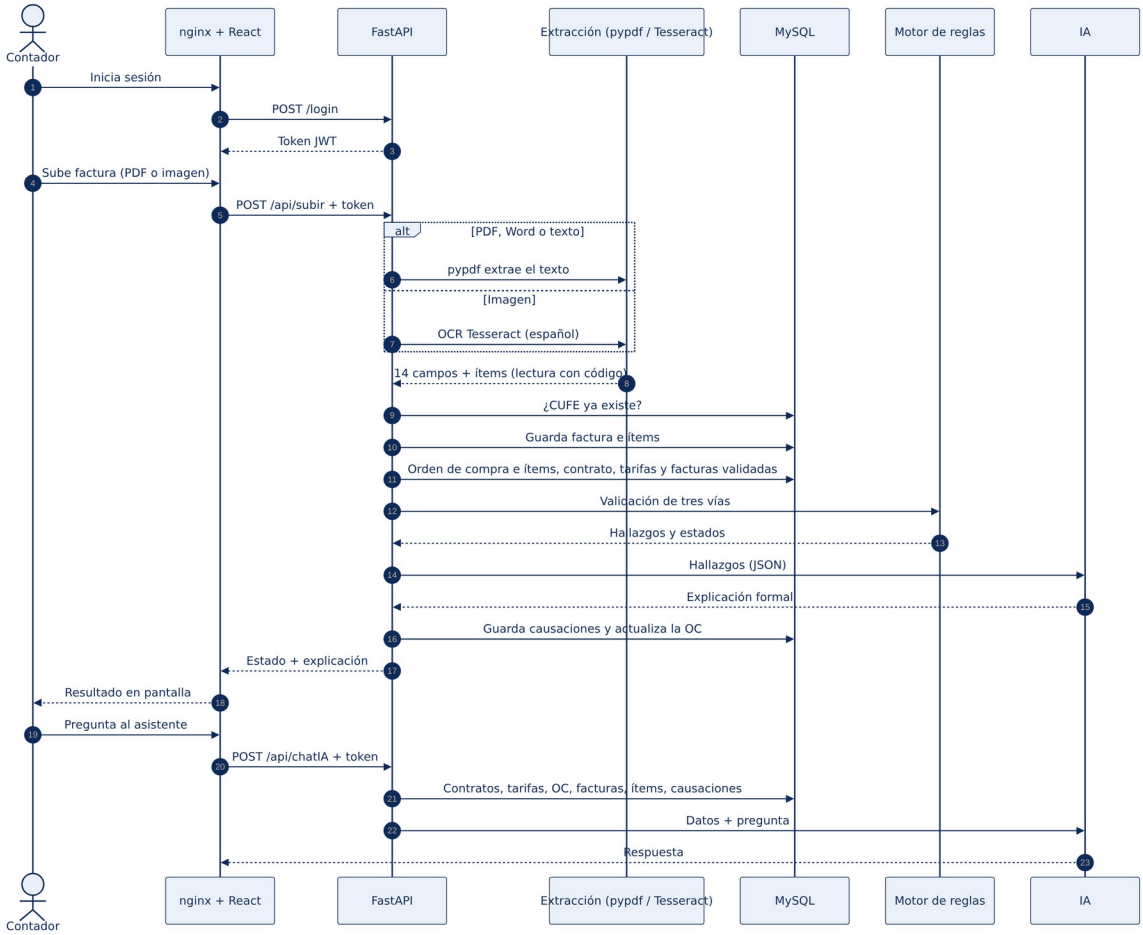


Figura 3. Diagrama de secuencia de carga y consulta

5. Motor de reglas y estados

Antes de validar, el sistema exige los 14 campos de la factura (emisor, NIT emisor, número, fechas de emisión y vencimiento, cliente, NIT cliente, orden de compra, moneda, contrato, subtotal, IVA, total y CUFE) y al menos un ítem. Si falta alguno responde con error y no procesa la factura.

Después se ejecuta la validación de tres vías (factura, orden de compra y contrato). Todas las reglas se revisan y cada incumplimiento genera un hallazgo; una factura puede tener varios.

Regla	Cómo se verifica	Estado si falla
Duplicado	El CUFE ya existe (la factura no se guarda)	Rechazada
Orden de compra	La OC citada en la factura existe y pertenece al mismo contrato	Rechazada
NIT en tres vías	El NIT coincide entre factura, orden de compra y contrato	Rechazada
IVA	IVA igual al 19% del subtotal	Con errores
Subtotal e IVA vs. OC	Subtotal e IVA de la factura iguales a los de la OC	Con errores
Ítems vs. contrato	Cada ítem está en las tarifas pactadas	Con errores
Precio vs. tarifa	Precio unitario no mayor que la tarifa máxima pactada	Requiere revisión
Ítems vs. OC	Cada ítem existe en la OC con el mismo precio y cantidad, y no falta ningún ítem de la OC	Con errores
Total vs. OC	Total de la factura igual al total de la OC	Rechazada
Tope acumulado	Contratos acumulados: facturas validadas más la actual no superan el valor total	Rechazada
Valor del período	Contratos mensuales, trimestrales o anuales: total de la factura y de la OC iguales al valor del período	Rechazada
Sin hallazgos	Todas las reglas se cumplen	Validada

Cada hallazgo genera un registro en la tabla causaciones con su estado y la explicación redactada por la IA. Si la factura queda validada, su orden de compra pasa a APROBADA; si no, a CANCELADA. En la interfaz, el semáforo muestra verde (validada), amarillo (requiere revisión) y rojo (con errores o rechazada).

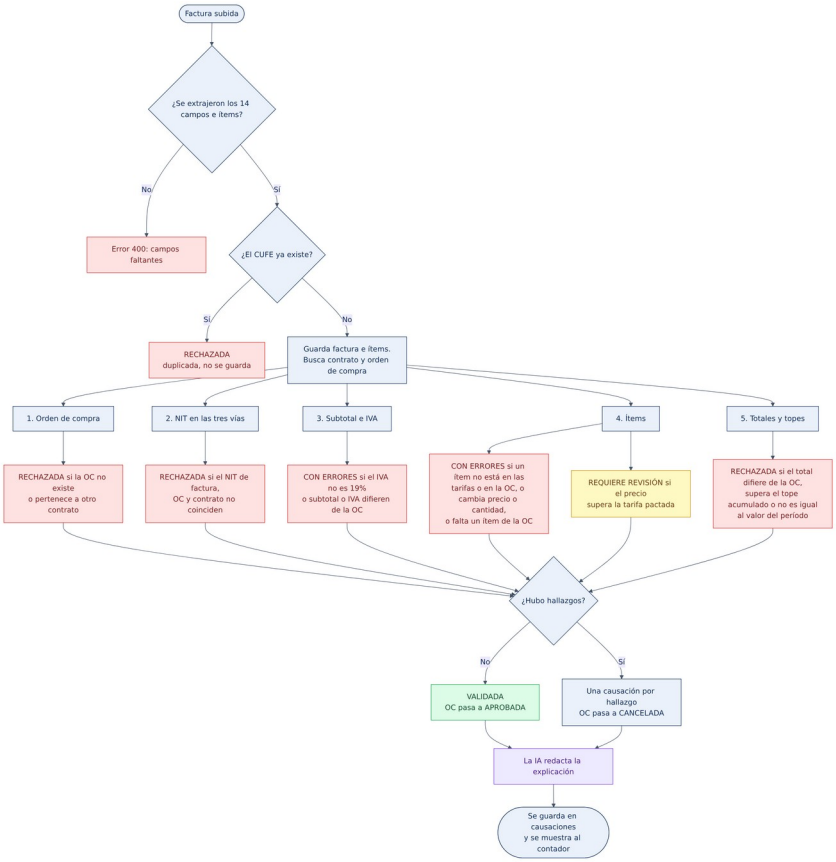


Figura 4. Árbol de decisión del motor de reglas

6. Uso de la inteligencia artificial

Lo hace el código	Lo hace la IA
Extraer texto de PDF (pypdf) e imágenes (Tesseract)	Redactar la explicación formal de cada hallazgo
Leer los campos de la factura	Responder preguntas del contador en el chat (/chatIA)
Cruce de tres vías: NIT, subtotal, IVA, ítems, cantidades, precios, totales y topes	
Actualizar el estado de la orden de compra	
Asignar el estado y guardar la causación	

- **Controles en los prompts:** la IA debe usar solo los datos recibidos, no inventar cifras, responder con formato fijo (Estado y Explicación) y temperatura 0,2 para respuestas estables.
- **Chat:** recibe en tiempo real los contratos, tarifas, órdenes de compra, facturas, ítems y causaciones de la base de datos y responde solo con esa información; si no la tiene, lo dice.
- **La IA no decide:** el estado ya viene calculado por el código cuando se le pide la explicación.

7. Modelo de datos

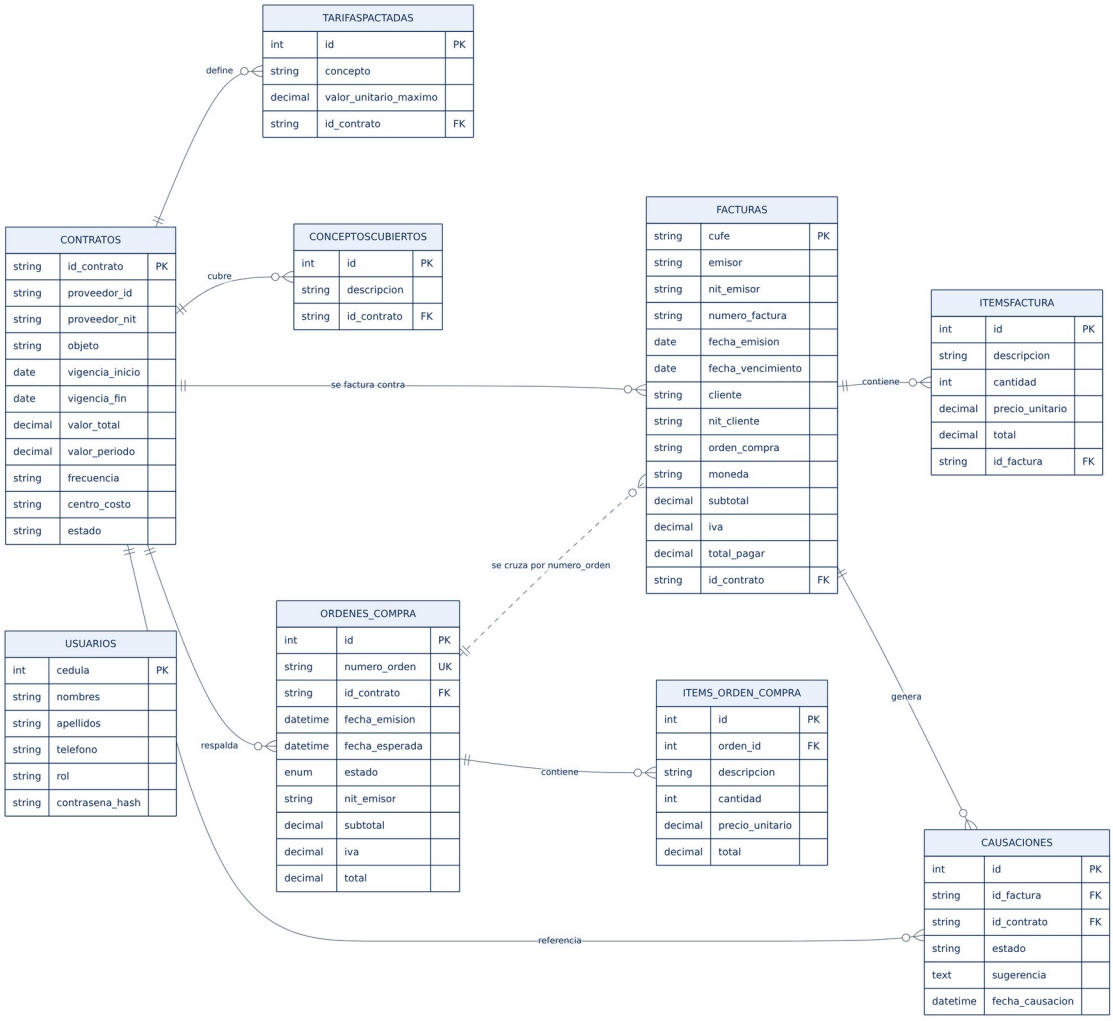


Figura 5. Modelo entidad-relación

8. API

Endpoint	Método	Uso
/login	POST	Inicia sesión con cédula y contraseña; devuelve token JWT
/subir	POST (form-data, campo archivo)	Procesa la factura y devuelve estado y explicación
/facturas	GET	Lista de facturas con su estado
/causaciones	GET	Todas las causaciones
/causaciones/factura/{cufe}	GET	Causaciones de una factura
/contratos	GET	Lista de contratos
/chatIA	POST (JSON: pregunta)	Respuesta del asistente
/heald	GET	Verificación de que el servidor está activo

Todas las rutas, excepto /login y /heald, exigen el token JWT. Detrás de nginx se llaman con el prefijo /api/.

9. Casos de uso

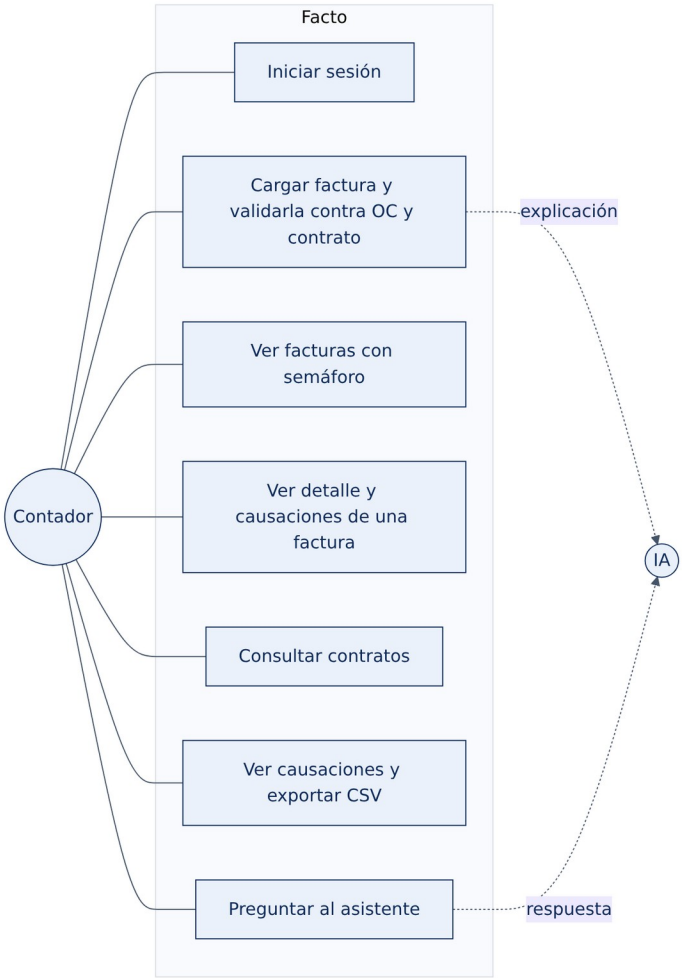


Figura 6. Casos de uso del contador

10. Despliegue

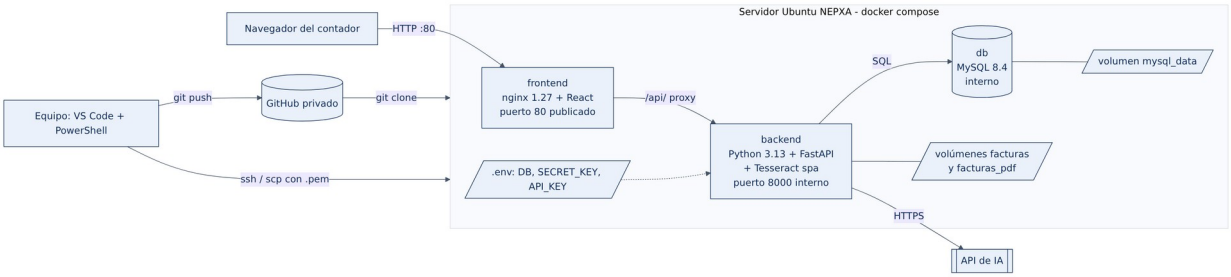


Figura 7. Diagrama de despliegue

1. **Configurar:** cp .env.example .env y completar DB_NAME, DB_USER, DB_PASSWORD, MYSQL_ROOT_PASSWORD, SECRET_KEY, ALGORITHM y API_KEY.
2. **Levantar:** docker compose up -d --build (el backend aplica las migraciones al arrancar).
3. **Datos y usuario:** cargar backend/datos_prueba.sql en el contenedor db y crear el primer usuario con docker compose exec backend python crear_admin.py.
4. **Usar:** abrir http://IP_DEL_SERVIDOR (puerto 80).

El paso a paso completo está en docs/DESPLIEGUE.md.

11. Seguridad y protección de datos

- **Autenticación:** inicio de sesión con contraseña cifrada con bcrypt y token JWT; la creación pública de usuarios está deshabilitada.
- **Red:** solo el puerto 80 está publicado; el backend y MySQL quedan en la red interna de Docker.
- **Contenedores:** el backend corre sin privilegios de root; nginx limita archivos a 12 MB y agrega cabeceras de seguridad.
- **Secretos:** credenciales y clave de IA en .env, fuera del repositorio.
- **Datos:** solo datos sintéticos; las facturas subidas se guardan en volúmenes de Docker.
- **Trazabilidad:** cada hallazgo queda en causaciones con estado, explicación y fecha, exportable a CSV.
- **Marco legal:** Ley 1581 de 2012 y cláusula de confidencialidad del hackathon.

12. Componentes de terceros

Componente	Licencia
React, Vite	MIT
FastAPI, SQLAlchemy, Alembic, PyMySQL, pypdf, PyJWT	MIT
Uvicorn	BSD-3-Clause
Tesseract OCR, pytesseract, cliente de la API de IA	Apache 2.0
Pillow	MIT-CMU (HPND)
nginx	BSD-2-Clause
MySQL Server	GPL v2 (servicio independiente en su propio contenedor)
API de IA	Servicio externo

13. Equipo

PySquad - Equipo 02: Nicole Stefany Velásquez Marín (líder), Sofia Moreno Salas, Harold Santiago Vergara Torres, Juan Manuel Quintero Lopez y Sergio Lamos Lozano. Mentor: Andrés Suárez.

Facto - Diagramas

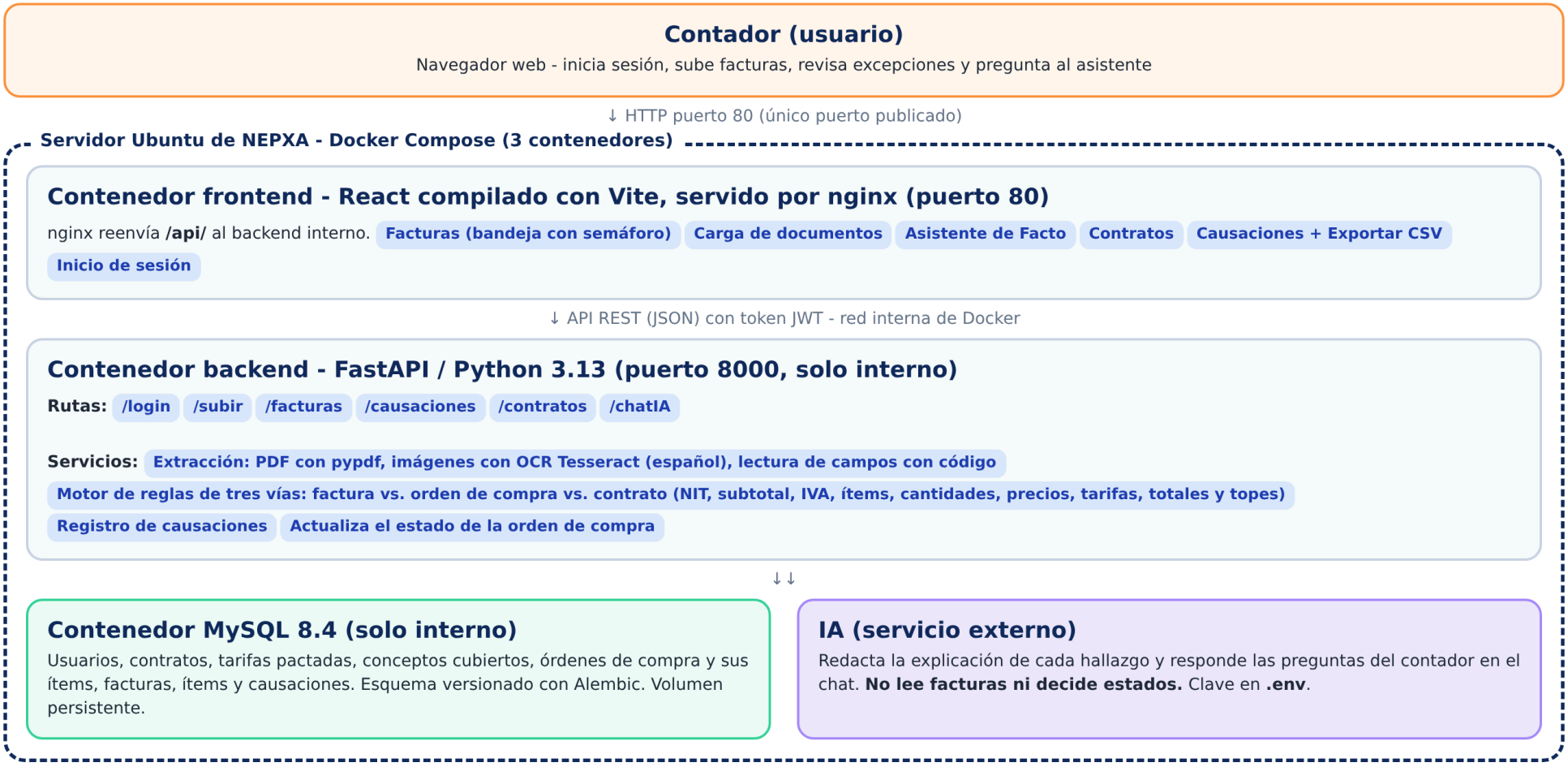
PySquad - Equipo 02 - Reto AI Invoice Processor - NEPXA AI Ops Challenge 2026

1. Arquitectura del sistema

Facto - PySquad

Facto - Arquitectura del sistema

PySquad - Equipo 02 - NEPXA AI Ops Challenge 2026



Principio: el código extrae y decide; la IA explica y responde; el contador revisa.

2. Flujo de funcionamiento

Facto - PySquad

Flujo de funcionamiento: de la factura a la causación

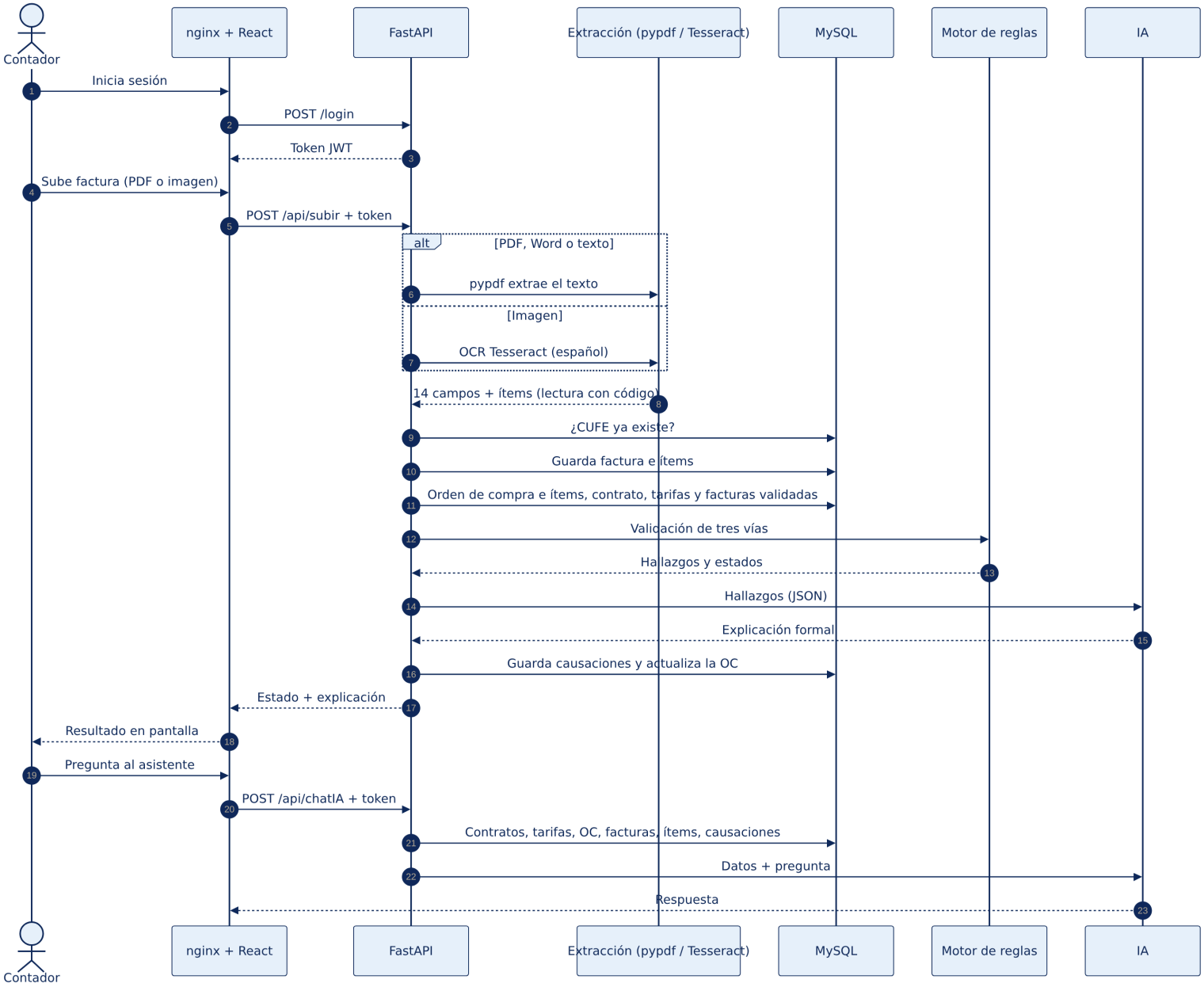
El código extrae y decide; la IA explica; el contador revisa.



Código calcula y decide IA explica y responde Persona sube y revisa

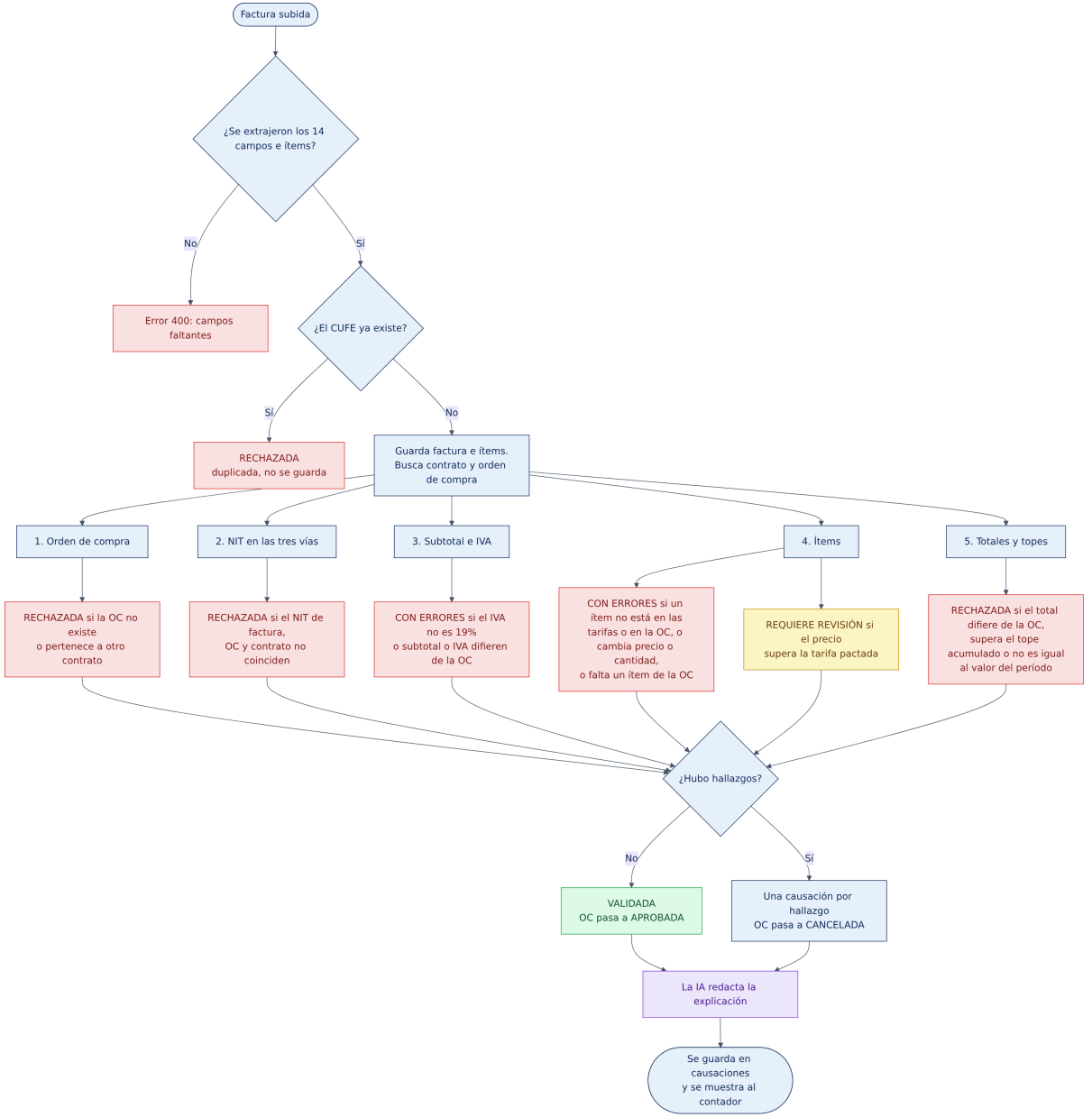
3. Diagrama de secuencia

Facto - PySquad



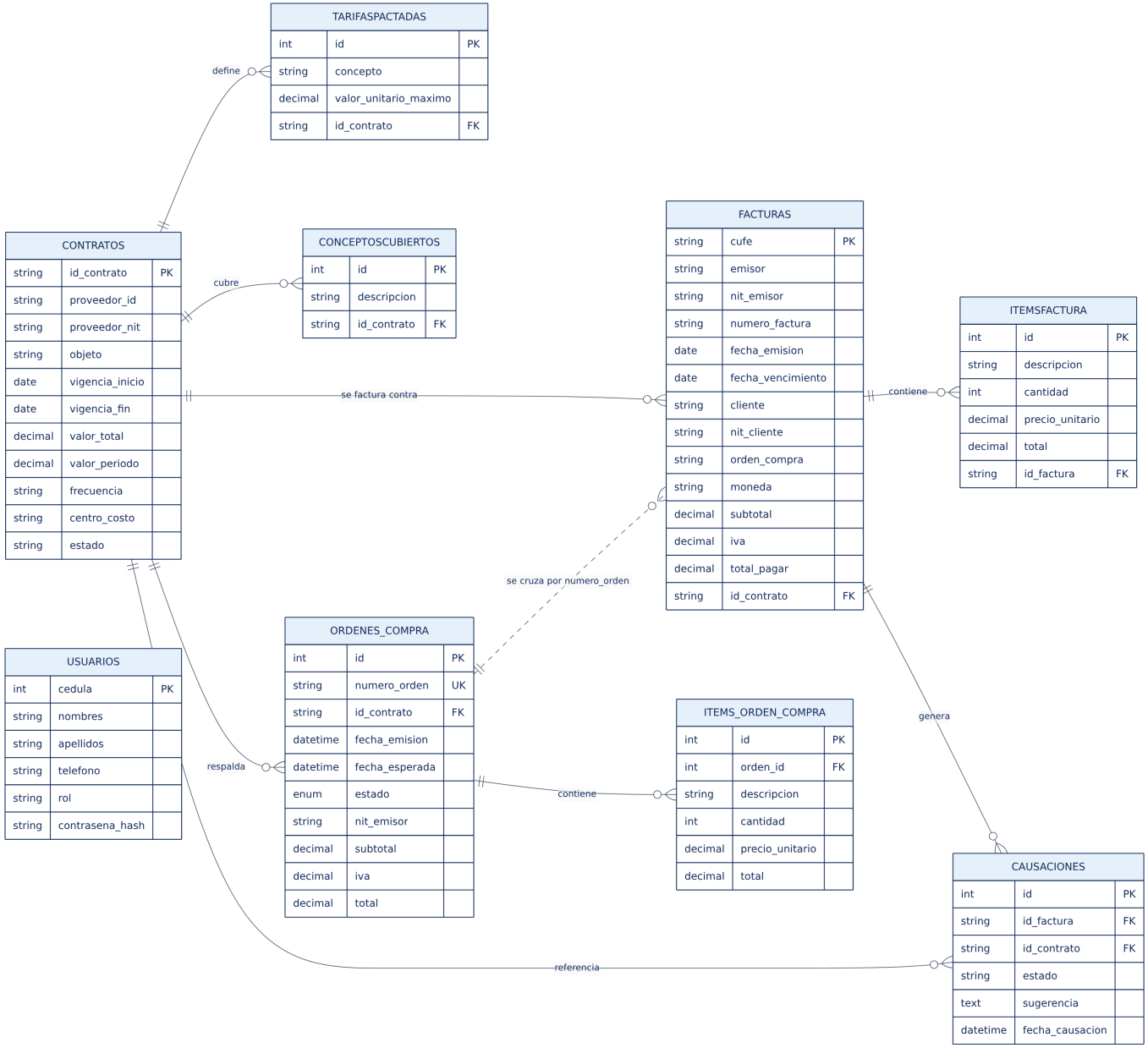
4. Árbol de decisión del motor de reglas (validación de tres vías)

Facto - PySquad



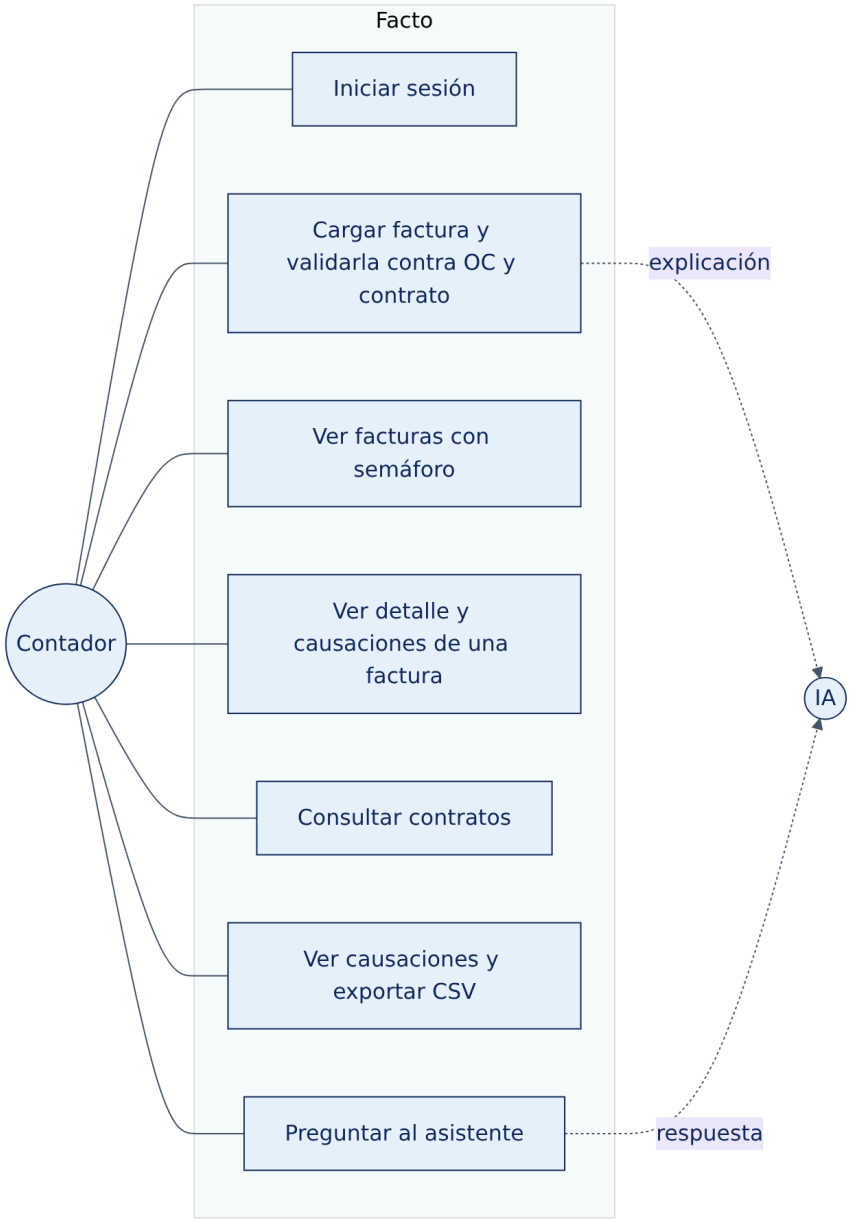
5. Modelo de datos

Facto - PySquad



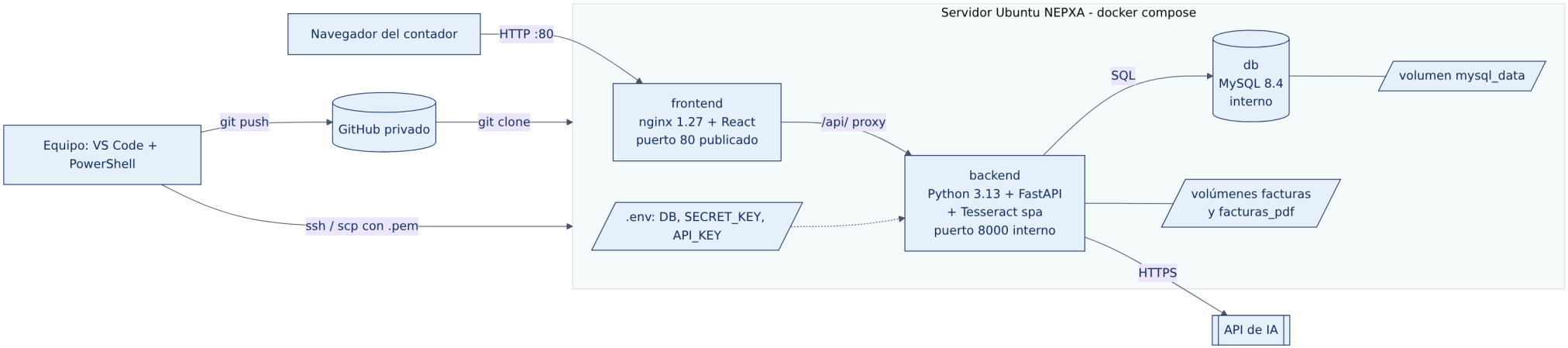
6. Casos de uso

Facto - PySquad



7. Diagrama de despliegue

Facto - PySquad



Facto - Cómo levantar el servicio en el servidor

Facto (PySquad, equipo 02) se despliega con **Docker Compose** en tres contenedores:

Contenedor	Imagen	Puerto	Función
`frontend`	nginx 1.27 + React compilado con Vite	80 (único publicado)	Sirve la aplicación y reenvía `/api/` al backend
`backend`	Python 3.13 + FastAPI + Tesseract (español)	8000 (solo interno)	API, extracción, motor de reglas, llamadas a IA
`db`	MySQL 8.4	3306 (solo interno)	Base de datos con volumen persistente

1. Requisitos en el servidor

- Docker y el plugin Docker Compose
- Acceso SSH con la llave del equipo

```
sudo apt update
sudo apt install -y docker.io docker-compose-v2 git
sudo usermod -aG docker $USER   # cerrar sesión y volver a entrar
```

2. Obtener el código

```
git clone https://github.com/tefvm1505/hackaton-pysquad.git
cd hackaton-pysquad
```

3. Variables de entorno

```
cp .env.example .env
nano .env
```

Variable	Qué es
`DB_NAME`, `DB_USER`, `DB_PASSWORD`	Base de datos y usuario de la aplicación
`MYSQL_ROOT_PASSWORD`	Clave del usuario root de MySQL
`SECRET_KEY`	Clave para firmar los tokens de sesión (JWT). Generar con `openssl rand -hex 32`
`ALGORITHM`	`HS256`
`API_KEY`	Clave de la API de IA

El archivo `.env` no se sube a git.

4. Construir y levantar

```
        docker compose up -d --build
        docker compose ps
```

Al arrancar, el backend aplica las migraciones de Alembic (`alembic upgrade head`) y levanta Uvicorn en el puerto 8000.

5. Cargar datos de prueba y crear el primer usuario

```
        docker compose exec -T db sh -c 'mysql -u"$MYSQL_USER" -p"$MYSQL_PASSWORD"
"$MYSQL_DATABASE"' < backend/datos_prueba.sql
        docker compose exec backend python crear_admin.py
```

`datos_prueba.sql` trae contratos, tarifas pactadas y conceptos cubiertos sintéticos. El endpoint de creación de usuarios está deshabilitado a propósito; el primer usuario se crea con `crear_admin.py`.

6. Verificar

1. Abrir `http://IP_DEL_SERVIDOR` e iniciar sesión con la cédula y contraseña creadas.
2. En **Carga de documentos**, subir una factura PDF de prueba: debe aparecer su estado y la explicación.
3. La factura aparece en **Facturas** y sus hallazgos en **Causaciones**.
4. Estado del backend: `docker compose exec backend python -c "import urllib.request;print(urllib.request.urlopen('http://localhost:8000/heald').read())"`

7. Operación

```
        docker compose logs -f backend      # ver registros del backend
        docker compose restart backend      # reiniciar el backend
        git pull && docker compose up -d --build   # actualizar a la última versión
        docker compose down                 # detener (los datos quedan en los volúmenes)
```

8. Seguridad del despliegue

- Solo el puerto 80 está expuesto; MySQL y el backend no se alcanzan desde fuera del servidor.
- El backend corre con un usuario sin privilegios de root.
- Contraseñas guardadas con bcrypt; sesiones con token JWT; todas las rutas, salvo `/login`, exigen token.
- nginx limita el tamaño de archivo a 12 MB y agrega cabeceras de seguridad.
- Secretos solo en `.env`; facturas subidas en volúmenes de Docker.

9. Subir la documentación al servidor (scp)

Desde PowerShell, en la carpeta donde están la llave y la carpeta `docs`:

```
        scp -i .\nepxa-hackathon-equipo-02.pem -r .\docs
ubuntu@IP_DEL_SERVIDOR:~/hackaton-pysquad/
```